Lab 4: Python Sensor Data Analysis

Google Colab, Filtering, Calibration, and Decisions

ENGR 1105 | Fall 2026

Name	**Partner**
Date	**Section**

Learning Objectives

1. use Google Colab to run Python without installing software;
2. create arrays and perform engineering calculations;
3. import sensor data from a CSV file;
4. make readable engineering plots;
5. identify invalid measurements and calculate a moving average;
6. fit a linear calibration model and calculate error; and
7. convert filtered sensor measurements into operating states.

1. Files and Software

You need a web browser, a Google account, the provided `sample_sensor_data.csv`, and the Week 4 Colab notebook. No paid software is required.

Save Before You Leave

Save the notebook to Google Drive or download the `.ipynb` file. A Colab runtime is temporary, and uploaded files may need to be uploaded again after a reset.

2. Lab Organization

Meeting	Main work
Lab 4.1	Colab basics, arrays, statistics, CSV import, and engineering plots.
Lab 4.2	Data cleaning, moving-average filtering, calibration, error analysis, decision states, and reflection.

Lab 4.1

Python Foundations and Engineering Plots

Week 4

3. Part A: Open Google Colab

1. Open Google Colab in a web browser and sign in.
2. Create a new notebook or open the provided Week 4 notebook.
3. Rename the notebook using your name and Lab 4.
4. Run the import cell below.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Colab Check

Item	Complete?	Notes
Notebook opened		
Imports run without error		
Notebook renamed/saved		

4. Part B: Variables and Calculations

Run:

```
voltage = 5.0
current = 0.12
power = voltage * current
print(power)
```

Record the result and unit:

_____

1. What does the symbol = do in Python?

   _____

2. What physical equation did the code evaluate?

   _____

5. Part C: NumPy Arrays

```
readings = np.array([20.6, 20.1, 19.8, 20.4, 20.2])
print(readings)
print(readings[0])
print(readings[1:4])
```

Array Results

Expression	Result
`readings[0]`	
`readings[1:4]`	
`len(readings)`	

Checkpoint 1

Explain why the first element is `readings[0]` rather than `readings[1]`.

6. Part D: Statistics

```
print(np.min(readings))
print(np.max(readings))
print(np.mean(readings))
print(np.std(readings))
```

Statistics

Quantity	Value	Interpretation
Minimum		
Maximum		
Mean		
Standard deviation		

7. Part E: Upload the CSV

Upload `sample_sensor_data.csv` using the Colab Files pane, then run:

```
df = pd.read_csv("sample_sensor_data.csv")
print(df.head())
print(df.columns)
print(df.shape)
```

How many rows and columns are in the file?

_____

8. Part F: Create an Engineering Plot

```
plt.plot(df["time_s"], df["raw_distance_cm"], "o-")
plt.xlabel("Time (s)")
plt.ylabel("Distance (cm)")
plt.title("Raw Ultrasonic Sensor Data")
plt.grid(True)
plt.show()
```

Plot Checklist

Check	Requirement
☐	Descriptive title
☐	x-axis label and unit
☐	y-axis label and unit
☐	Data markers or readable line
☐	Plot is large enough to interpret

Checkpoint 2

Describe one pattern or suspicious value visible in the raw-data plot.

Lab 4.2

Filtering, Calibration, and Reliable Decisions

Week 4

9. Part G: Identify Invalid Measurements

Run:

```
valid = (
    (df["raw_distance_cm"] > 1) &
    (df["raw_distance_cm"] < 400)
)

clean = df.loc[valid].copy()
print(clean)
```

How many rows were removed?

_____Why should the original `df` remain unchanged?

_____

_____

10. Part H: Moving Average

```
clean["filtered_cm"] = (
    clean["raw_distance_cm"]
    .rolling(3, center=True, min_periods=1)
    .mean()
)
```

Plot raw and filtered data on the same axes:

```
plt.plot(clean["time_s"], clean["raw_distance_cm"],
         "o-", label="Raw")
plt.plot(clean["time_s"], clean["filtered_cm"],
         "-", label="Filtered")
plt.xlabel("Time (s)")
plt.ylabel("Distance (cm)")
plt.legend()
plt.grid(True)
plt.show()
```

Filter Comparison

Feature	Raw data	Filtered data
Short-term variation		
Response to changes		
Easier to interpret?		

11. Part I: Compare Filter Window Sizes

Repeat the filter with windows of 3 and 5 samples.

```
clean["avg3"] = clean["raw_distance_cm"].rolling(
    3, center=True, min_periods=1).mean()
clean["avg5"] = clean["raw_distance_cm"].rolling(
    5, center=True, min_periods=1).mean()
```

Window Tradeoff

Question	Window 3	Window 5
Which is smoother?		
Which follows changes faster?		
Which would you choose?		

Explain the tradeoff between smoothing and response speed.

12. Part J: Linear Calibration

```
m, b = np.polyfit(
    clean["raw_distance_cm"],
    clean["reference_distance_cm"],
    1
)

clean["calibrated_cm"] = (
    m * clean["raw_distance_cm"] + b
)

print(m, b)
```

Record the calibration model:

$$\text{reference distance} = (\qquad)(\text{raw distance}) + (\qquad)$$

What does the slope m do to the raw reading?

What does the intercept b do?

13. Part K: Evaluate Error

```
raw_error = (
    clean["raw_distance_cm"] -
    clean["reference_distance_cm"]
)
cal_error = (
    clean["calibrated_cm"] -
    clean["reference_distance_cm"]
)

raw_mae = np.mean(np.abs(raw_error))
cal_mae = np.mean(np.abs(cal_error))

print(raw_mae, cal_mae)
```

Error Results

Metric	Value (cm)	Interpretation
Raw MAE		
Calibrated MAE		

Did calibration improve this data set? Support your answer with the MAE values.

Checkpoint 3

Why is an error metric stronger evidence than saying that a calibration plot "looks better"?

14. Part L: Classify Operating States

```
def classify(distance_cm):
    if distance_cm < 15:
        return "DANGER"
    elif distance_cm < 35:
        return "WARNING"
    return "SAFE"

clean["state"] = clean["filtered_cm"].apply(classify)
print(clean[["filtered_cm", "state"]])
```

Threshold Meaning

State	Physical interpretation
DANGER	
WARNING	
SAFE	

15. Part M: Results Table

Display only the most important columns:

```
results = clean[[
    "time_s",
    "raw_distance_cm",
    "filtered_cm",
    "calibrated_cm",
    "state"
]]

print(results.round(2))
```

Select one row and record it:

Time	Raw	Filtered	Calibrated	State

16. Part N: Design Challenge

Complete one:

☐ **Filter comparison:** Plot windows 1, 3, and 5 and justify the preferred window.

☐ **New decision zones:** Choose new DANGER, WARNING, and SAFE thresholds for a different robot scenario and explain them.

☐ **Week 3 data:** Replace the sample CSV with your own Week 3 ultrasonic data and repeat the analysis.

☐ **Calibration plot:** Plot raw reading on the x-axis, reference distance on the y-axis, and overlay the fitted calibration line.

Describe what you changed and what you learned.

Checkpoint 4

Explain the complete data pipeline: Arduino measurement → CSV → Python analysis → engineering decision.

17. Final Reflection

1. What advantage did the plot provide compared with reading values one at a time in Serial Monitor?

2. What did the moving average improve, and what did it sacrifice?

3. Did calibration improve the sensor data? Give numerical evidence.

4. How are thresholds related to the controller behavior that you will build in later weeks?

5. Name one way that incorrect data cleaning or plotting could lead to a bad engineering decision.

18. Submission Checklist

Check	Required item
☐	Completed Lab 4 worksheet
☐	Saved/downloaded Colab notebook (`.ipynb`)
☐	Raw-data engineering plot
☐	Raw-versus-filtered plot
☐	Calibration coefficients and error results
☐	Operating-state results
☐	One completed design challenge